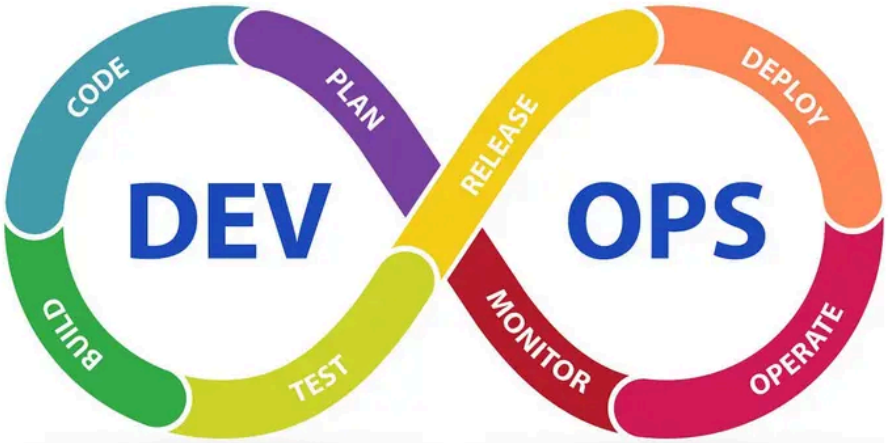
17 Django-marcador

¿Qué es DevOps?

DevOps es un conjunto de prácticas, herramientas y una filosofía cultural que une a los equipos de desarrollo (Dev) y operaciones de TI (Ops) para automatizar e integrar sus procesos. Su objetivo es acortar el ciclo de desarrollo, aumentando la velocidad, calidad y confiabilidad en la entrega de software.



En esta web técnica podemos encontrar más información.

Si ya sabemos construir aplicaciones, el siguiente paso lógico es saber **dónde y cómo se mantienen vivas**. Python es el lenguaje rey en la infraestructura moderna.

1. Fundamentos de Contenedores (Docker)

Antes de tocar código, debemos entender que un contenedor no es una máquina virtual, sino un proceso aislado con todo lo necesario para ejecutarse.

- **El Dockerfile:** Cómo escribir la "receta" para una app de API (Imagen base Python, requirements.txt, variables de entorno).
- **Imágenes vs. Contenedores:** Construcción (build) y ejecución (run).
- **Gestión de Volúmenes:** Para que los datos de la base de datos no desaparezcan al apagar el contenedor.

2. Orquestación Local (Docker Compose)

17 Django-marcador

Sabemos hacer APIs, pero estas suelen necesitar una base de datos (PostgreSQL, MySQL) o un sistema de caché (Redis).

- **El archivo docker-compose.yml:** Levantar la API y la Base de Datos con un solo comando.
- **Redes en Docker:** Cómo se comunican los servicios entre sí sin usar IPs locales.

3. CI/CD (Integración y Despliegue Continuo)

El código se testea y se "empaqueta" solo al subirlo a GitHub.

- **GitHub Actions:** Crear un flujo que ejecute Pytest automáticamente.
- **Docker Hub:** Subir la imagen final a un registro público/privado.

Más sobre Docker.

Usaremos este proyecto como ejemplo real de un flujo DevOps. Tenemos una API con **FastAPI** conectada a **PostgreSQL**, corriendo dentro de **Docker** para evitar el típico 'en mi máquina funciona'. Pero lo más importante es la automatización: incluimos tests que **GitHub Actions** ejecuta por nosotros cada vez que subimos código a GitHub. Es nuestra red de seguridad: si una modificación introduce un error, el 'robot' de GitHub nos avisará antes de que el problema pase a mayores.

Además, gracias a FastAPI, el proyecto autogenera documentación interactiva (Swagger), facilitando que otros desarrolladores prueben los endpoints sin necesidad de software externo.

Requisitos Previos

Antes de comenzar, asegúrate de tener instaladas las siguientes herramientas en tu entorno de desarrollo:

1. Software Fundamental

- **Docker Desktop:** Imprescindible para levantar los contenedores de la API y la base de datos.
- **Docker Compose:** Generalmente viene incluido con Docker Desktop. Es el encargado de orquestar los múltiples servicios.
- **Git:** Necesario para clonar el repositorio y gestionar las versiones que luego leerá GitHub Actions.

- **Python 3.11+:** Aunque la app correrá dentro de Docker, es recomendable tenerlo en local para el autocompletado de tu editor (IDE) y la gestión de extensiones.

2. Configuración en Windows

- **WSL2 (Windows Subsystem for Linux):** Para un rendimiento óptimo de Docker en Windows, se recomienda encarecidamente usar el motor de WSL2.

3. Cuentas y Accesos

- **Cuenta en GitHub:** Para alojar el código y ejecutar los flujos de trabajo de **GitHub Actions**.

Instalación de WSL2 y Docker Desktop

Un tutorial de instalación de WSL2 y Docker Desktop lo tenemos en este enlace.

Tecnologías Utilizadas

El proyecto se apoya en un stack moderno de tecnologías diseñadas para ofrecer alto rendimiento y facilidad de mantenimiento.

Backend & Lenguaje

- **Python 3.11+**: Lenguaje base por su versatilidad y ecosistema.
- **FastAPI**: Framework de alto rendimiento para construir APIs. Destaca por su rapidez, tipado automático y documentación interactiva bajo estándares OpenAPI.

Base de Datos & Persistencia

- **PostgreSQL**: Sistema de base de datos relacional de nivel empresarial, conocido por su robustez y fiabilidad.
- **SQLAlchemy**: El ORM (Object-Relational Mapping) que permite interactuar con la base de datos usando objetos de Python en lugar de consultas SQL puras.

17 Django-marcador

Validación & Seguridad

- **Pydantic**: Encargado de la validación de datos. Asegura que los datos que entran y salen de la API cumplen estrictamente con los formatos definidos.

Infraestructura & Despliegue (DevOps)

- **Docker**: Tecnología de contenedores que empaqueta la app para que se ejecute idénticamente en cualquier servidor.
- **Docker Compose**: Herramienta para definir y correr aplicaciones multi-contenedor (API + Base de Datos).

Calidad & Automatización (CI/CD)

- **Pytest**: Framework de pruebas para verificar que cada parte del código funciona como se espera.
- **GitHub Actions**: Plataforma de integración continua que ejecuta la batería de tests automáticamente en cada cambio de código.

Comandos de uso frecuente

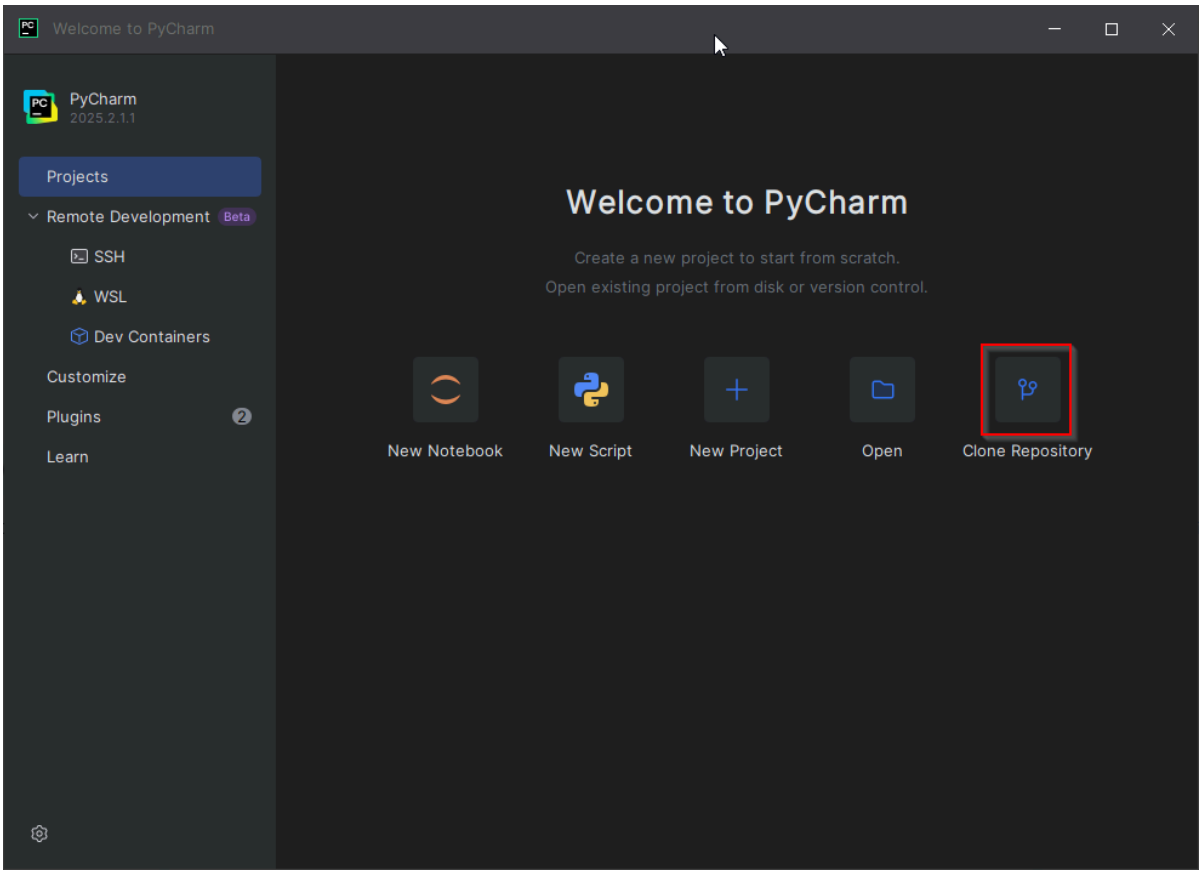
- `docker-compose up --build`: Construye las imágenes y levanta todo el sistema.
- `docker-compose down`: Apaga todo y limpia la red.
- `docker-compose ps`: Para ver si los dos contenedores están "Up" (vivos).
- `docker-compose exec api pytest -s -p no:cacheprovider tests/`: para ejecutar testeos (se verá con detalle más tarde)

más comandos docker

Anatomía del Proyecto: "API DevOps"

Instalamos el proyecto:

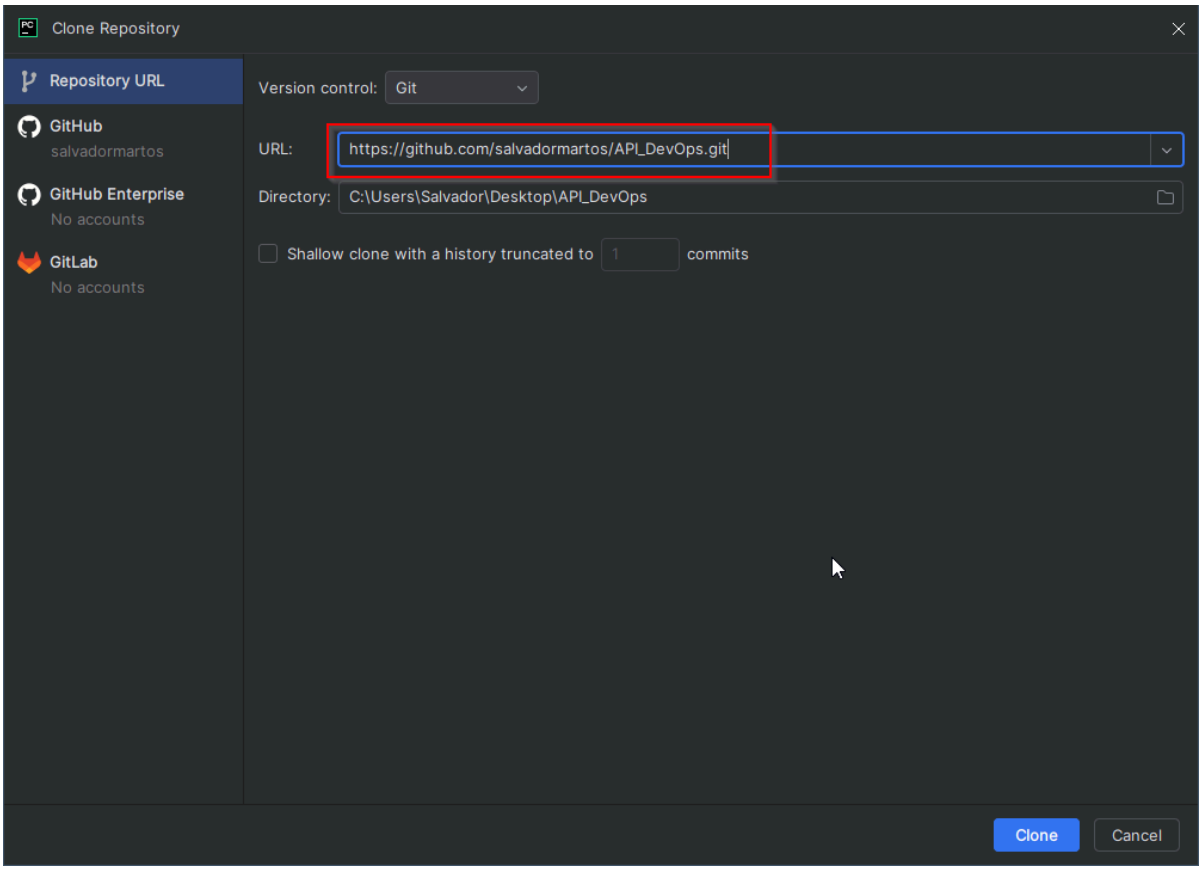
17 Django-marcador

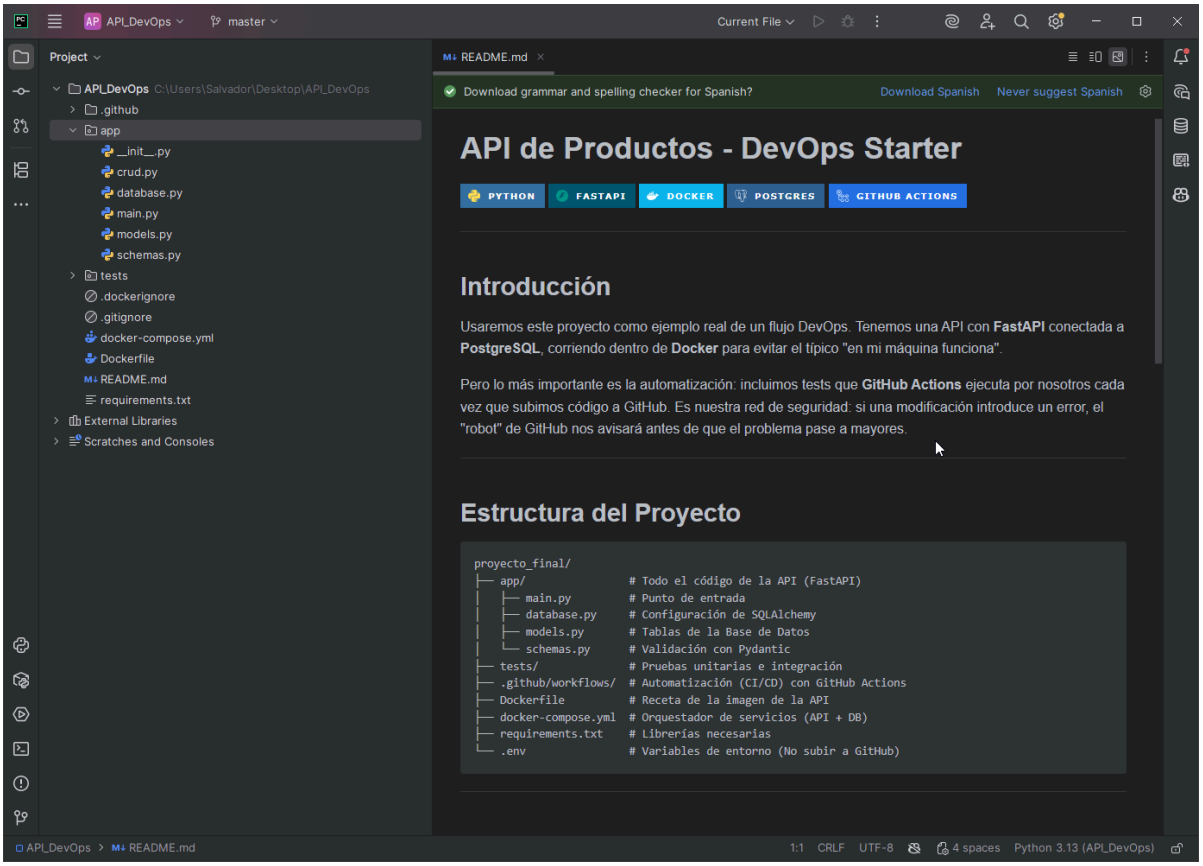


clonamos el proyecto:

https://github.com/salvadormartos/API_DevOps.git

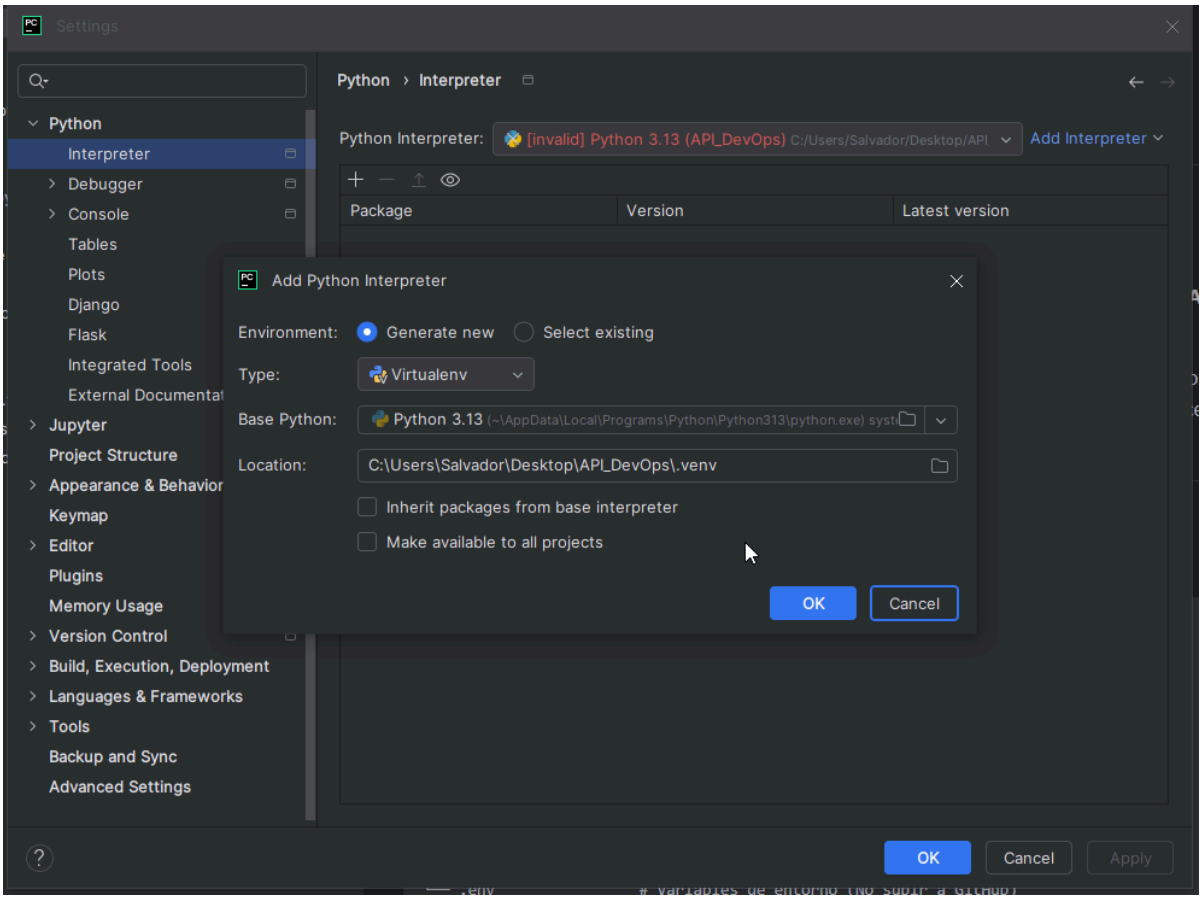
17 Django-marcador





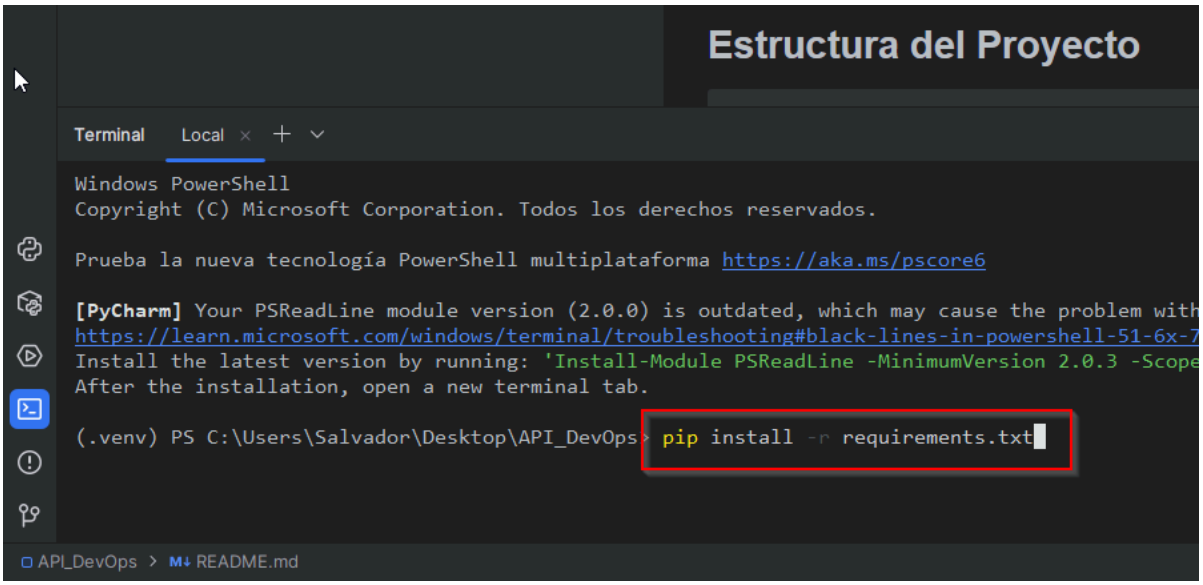
17 Django-marcador

Creamos el entorno virtual.ve a **File > Settings** (o *PyCharm > Settings* en Mac), selecciona Project: > Python Interpreter, haz clic en el engranaje, elige Add..., selecciona Virtualenv Environment, y crea uno Nuevo en la carpeta de tu proyecto

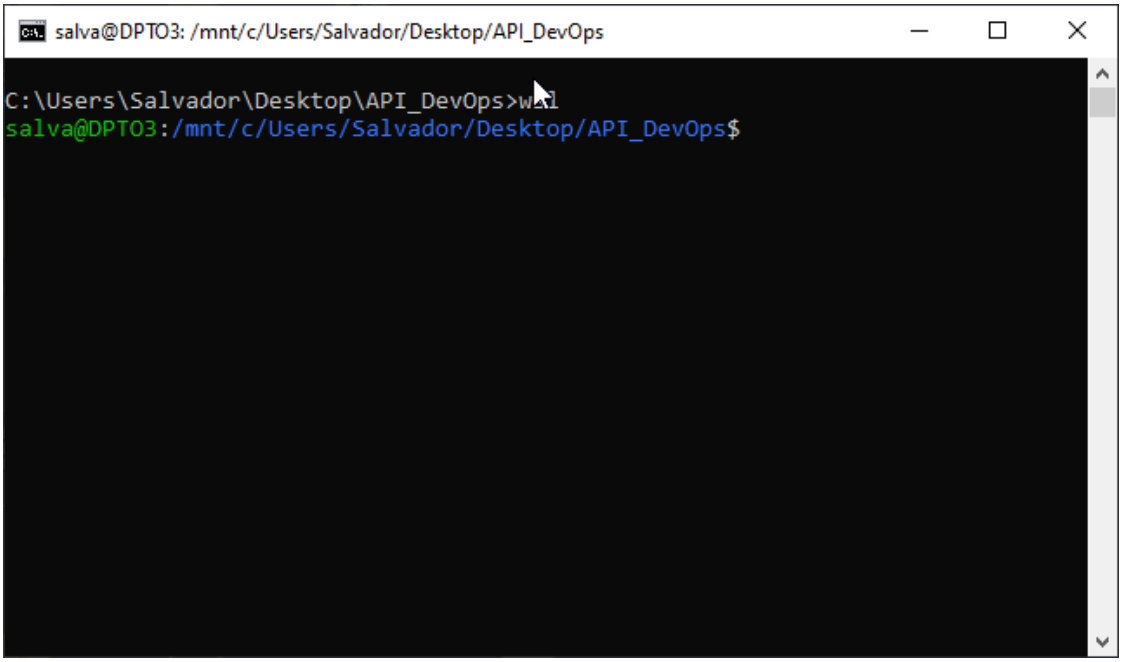


Instala todas las bibliotecas, paquetes y versiones específicas necesarias para que el proyecto funcione correctamente:

17 Django-marcador

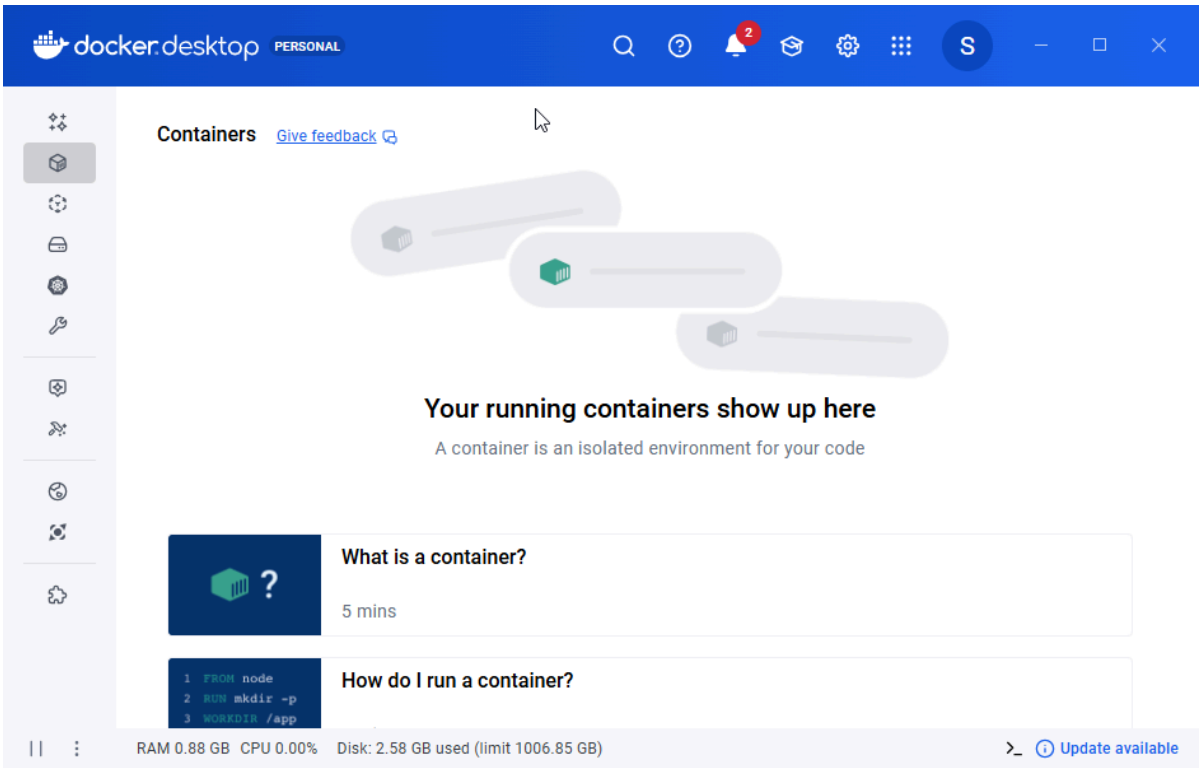


Inicia WSL2:



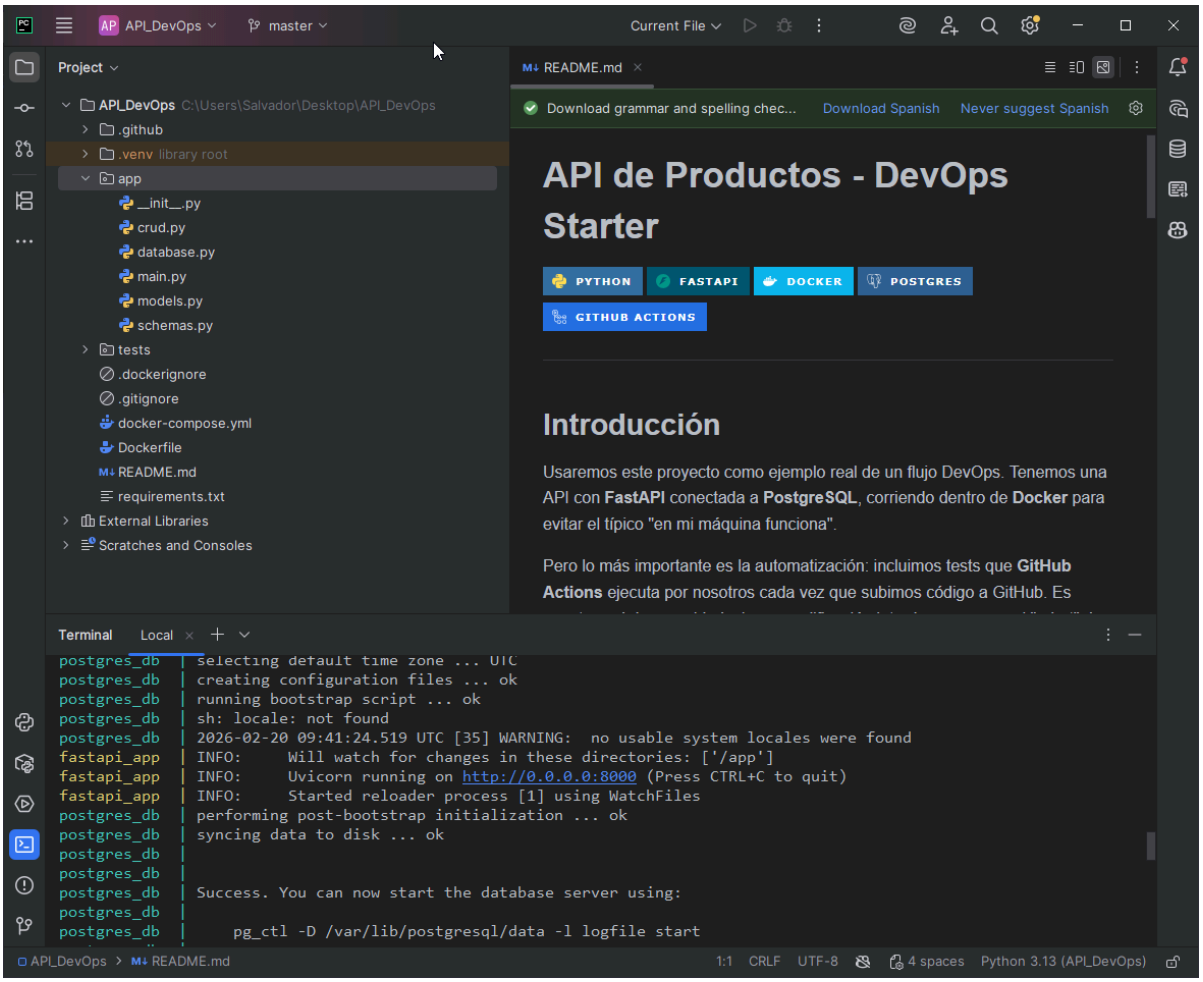
Inicia Docker:

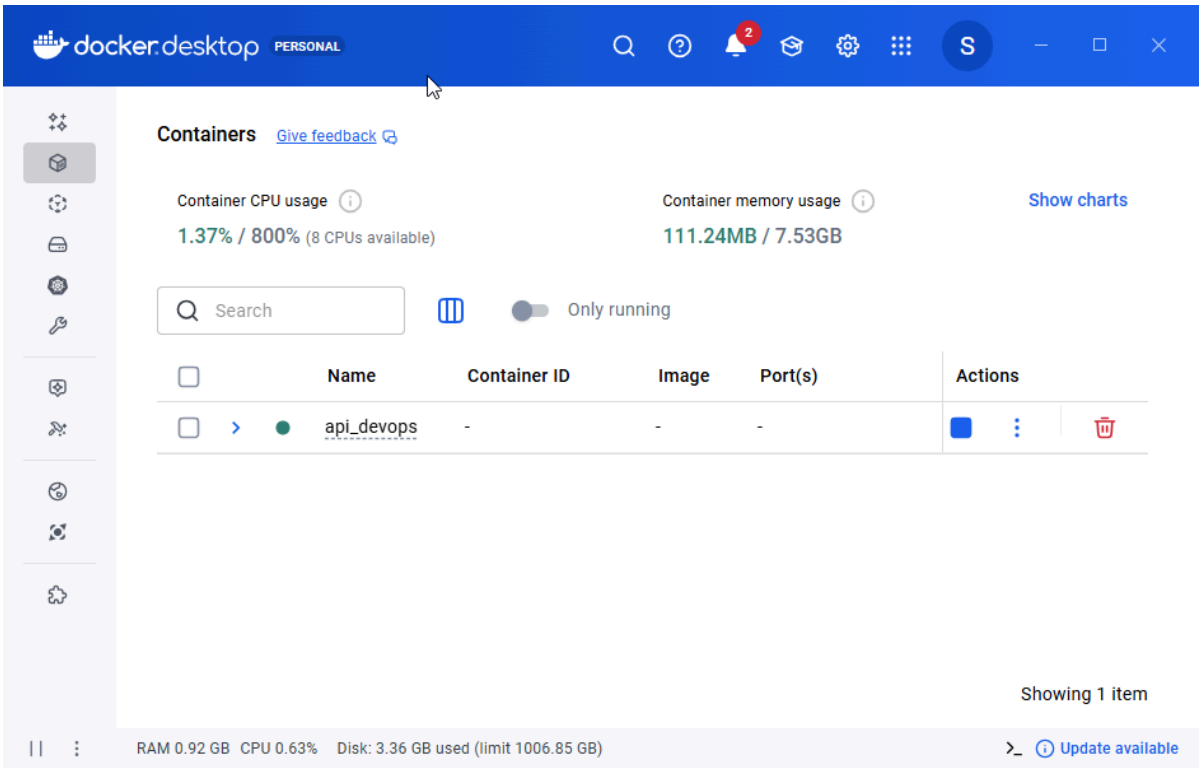
17 Django-marcador



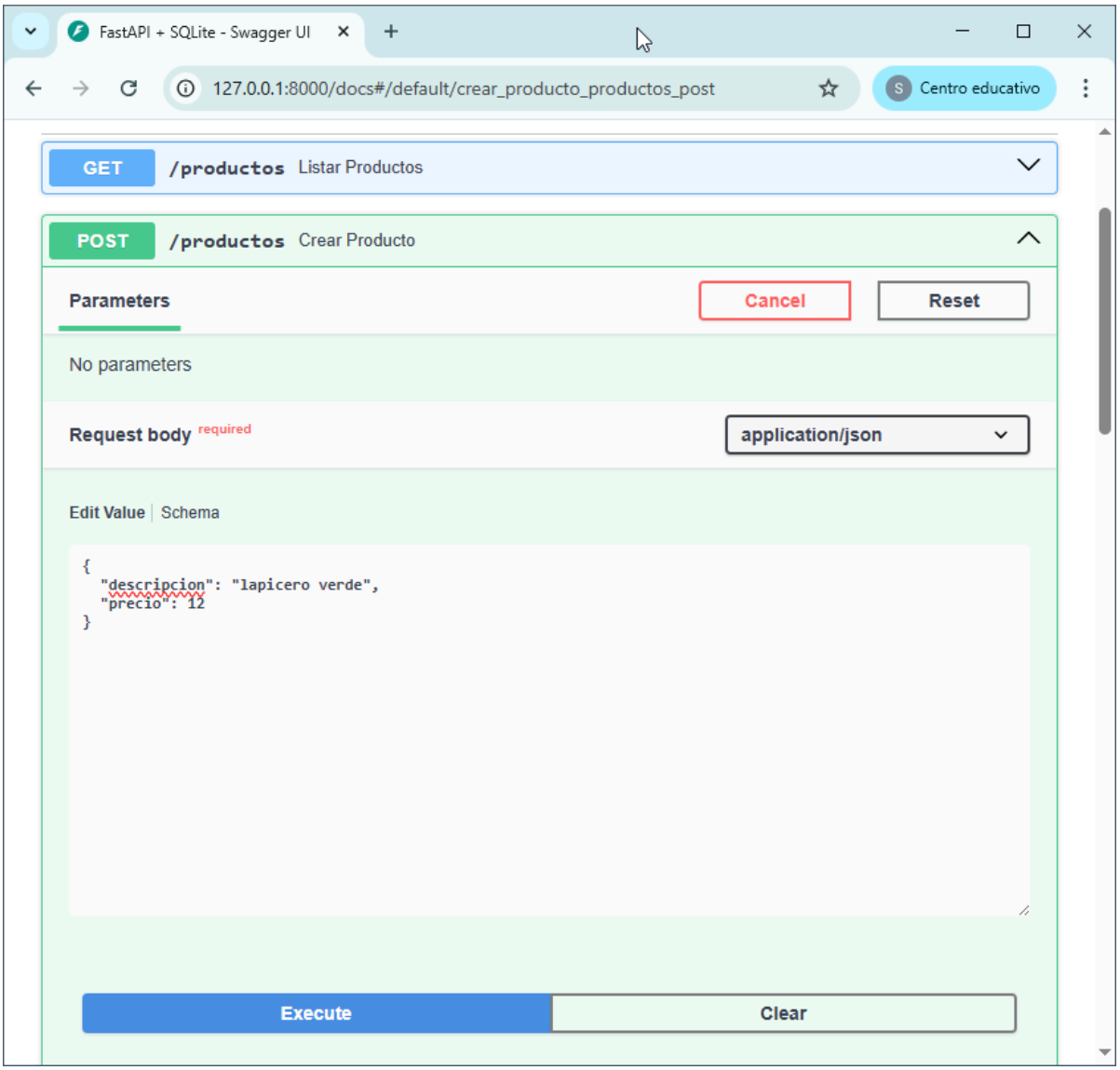
ejecuta: `docker-compose up --build`

17 Django-marcador



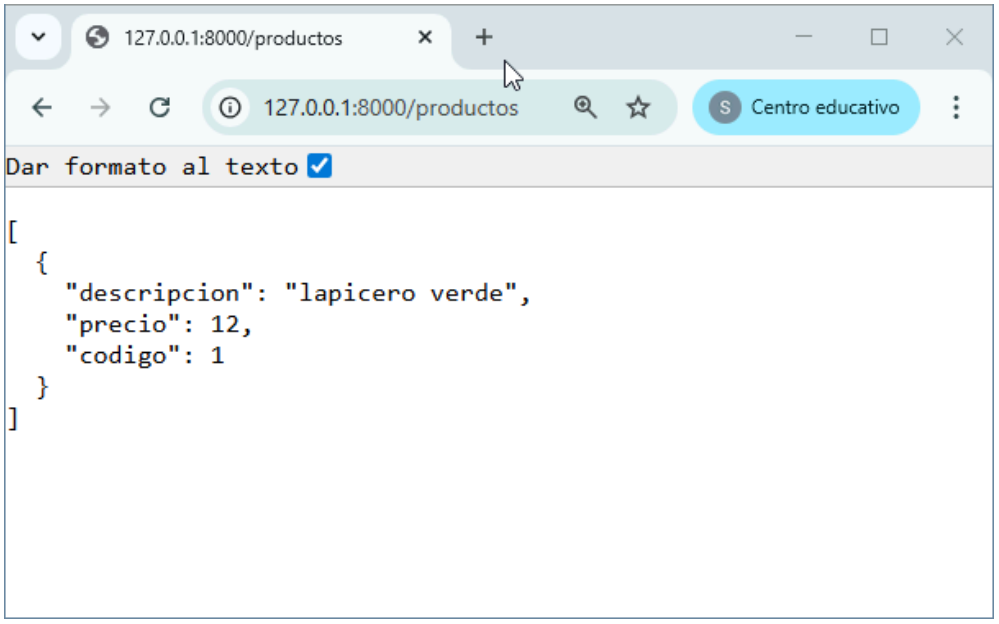


Creamos un producto:



Vemos el producto creado:

17 Django-marcador



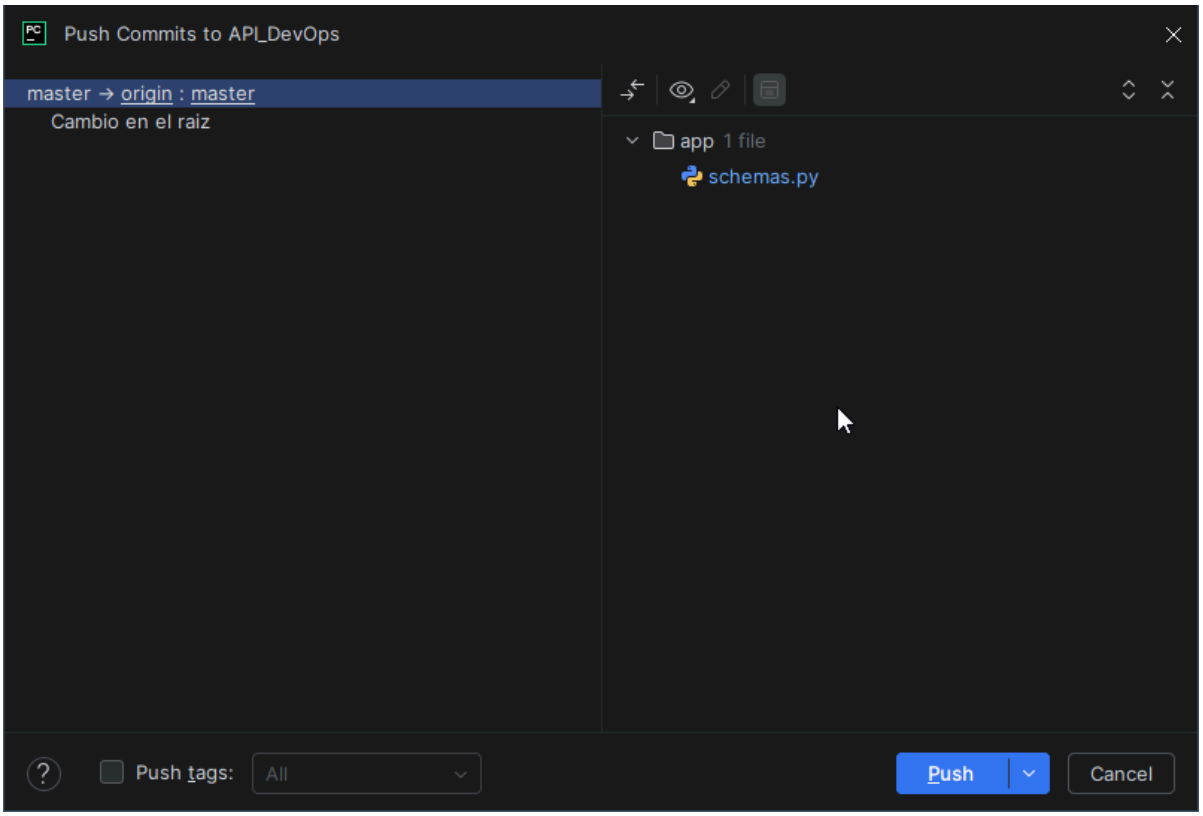
Si en la codificación de nuestro proyecto cometemos un error como puede ser este:

schemas.py

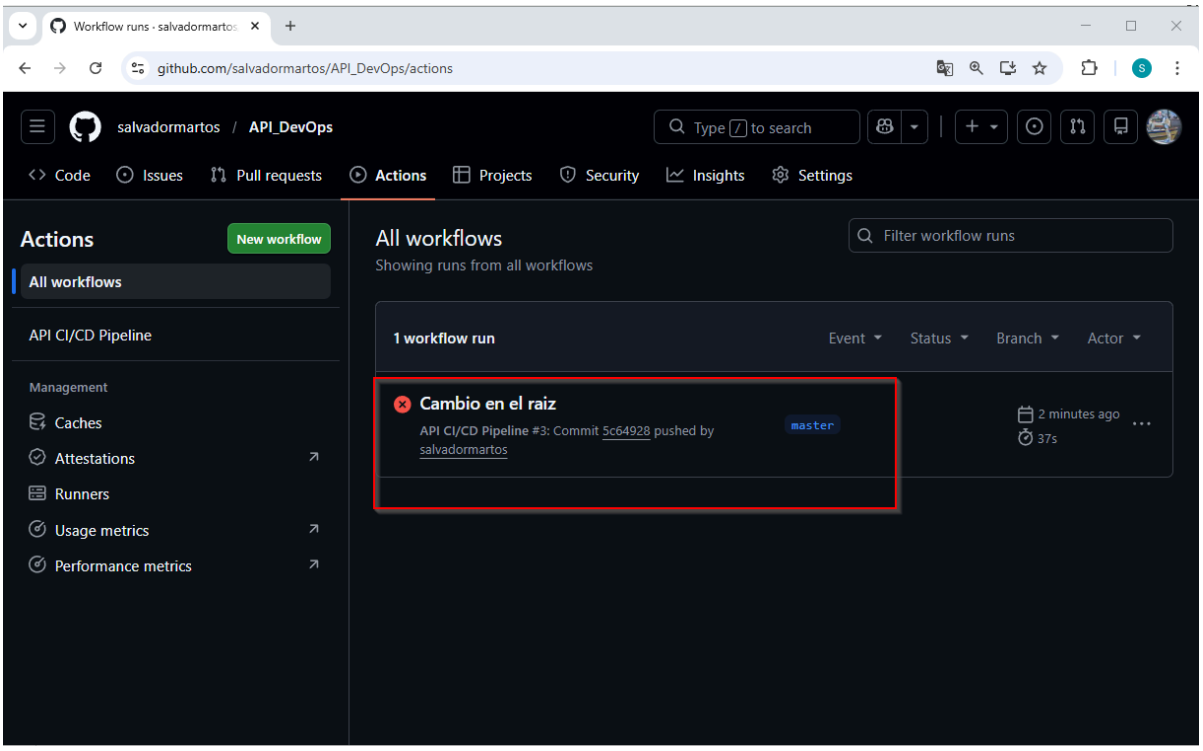
```
class ProductoBase(BaseModel):
   descripcion: str = Field(min_length=3, max_length=100)
   #precio: float = Field(gt=0, description="El precio debe ser mayor que cero")
   precio: float = Field()
```

donde no contemplamos que el precio debe de ser mayor que cero, cuando subimos el código a Github nuestros test lo van a detectar:

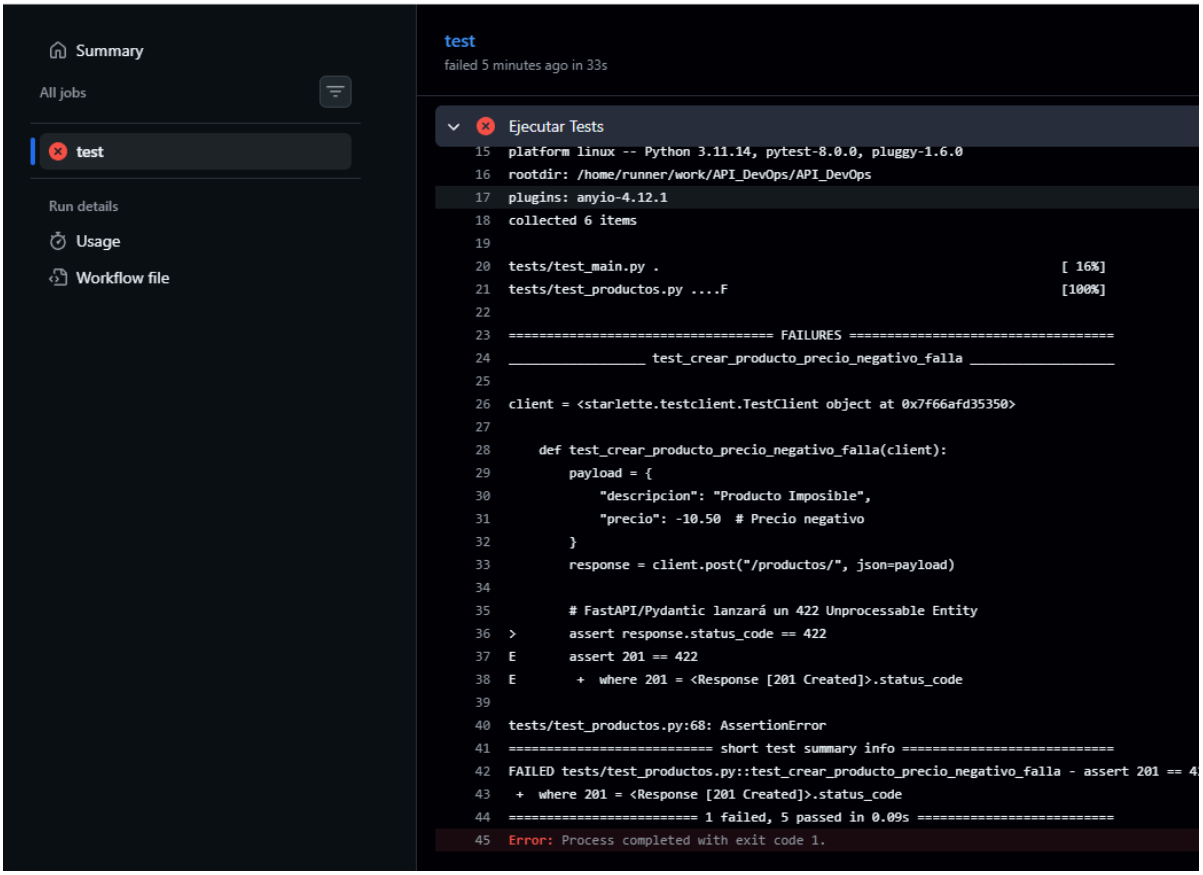
17 Django-marcador



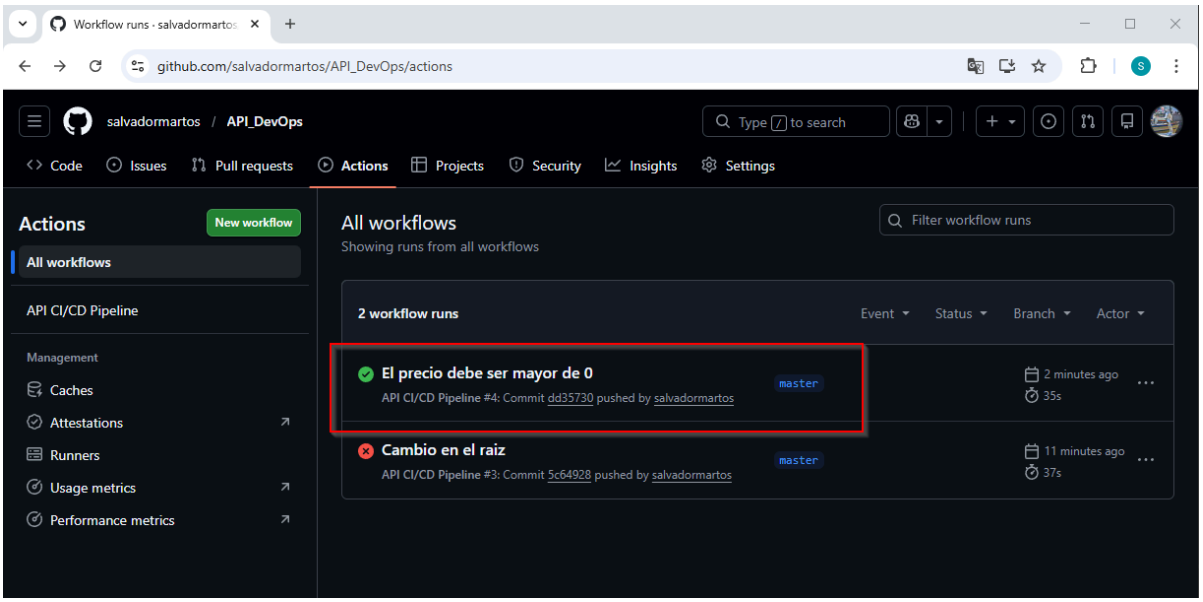
en Github Actions:



17 Django-marcador



Si corregimos el error y subimos el código a Github:



Vamos a ver un poco todo esto:

17 Django-marcador

Para que cualquier desarrollador entienda nuestro proyecto, vamos a dividir la estructura en tres capas: **Aplicación**, **Infraestructura** y **Automatización**.

1. La Capa de Aplicación (Carpeta app/)

Es el corazón del negocio. Aquí reside la lógica de Python y FastAPI.

- `__init__.py`: Convierte la carpeta en un módulo de Python, permitiendo importaciones limpias como `from app import models`.
- `main.py`: El director de orquesta. Aquí se definen las rutas (endpoints) y se arranca la aplicación con Uvicorn.
- `database.py`: El puente hacia los datos. Gestiona la conexión con PostgreSQL y el ciclo de vida de las sesiones de la base de datos.
- `models.py`: Define cómo se ven las tablas en la base de datos usando **SQLAlchemy** (POO).
- `schemas.py`: Define cómo viajan los datos por la red usando **Pydantic**. Valida que lo que el usuario envía sea correcto antes de que toque la base de datos.

2. La Capa de Infraestructura (Docker)

Define cómo se construye y se ejecuta el sistema en cualquier ordenador del mundo.

- `Dockerfile`: La receta paso a paso para crear la imagen de tu API. Asegura que Python 3.11 y las librerías sean siempre las mismas.
- `docker-compose.yml`: El orquestador. Permite que la API y la base de datos Postgres nazcan, se comuniquen y mueran juntas con un solo comando.
- `.dockerignore`: Evita que archivos pesados o sensibles (como `.venv` o carpetas de caché) se suban a la imagen de Docker, manteniéndola ligera y segura.
- `.env`: El baúl de los secretos. Aquí guardas contraseñas y URLs de conexión. **Regla de oro:** Jamás se sube a GitHub para evitar filtraciones de seguridad.

3. La Capa de Calidad y Automatización

Donde nos aseguramos de que el código no se rompa al subir cambios.

- `tests/`: Tu red de seguridad. Contiene las pruebas que verifican que, por ejemplo, un precio negativo sea rechazado o que un producto se guarde bien.

17 Django-marcador

- **.github/workflows/ci.yml**: Tu "robot" de confianza. Cada vez que haces un git push, este archivo le dice a GitHub: "Levanta un servidor, instala todo y dime si los tests pasan".

El archivo docker-compose.yml

```yaml
services:
 db:
   image: postgres:15-alpine
   container_name: postgres_db
   restart: always
   environment:
     - POSTGRES_USER=salva
     - POSTGRES_PASSWORD=salva
     - POSTGRES_DB=productos
   ports:
     - "5432:5432"
   volumes:
     - postgres_data:/var/lib/postgresql/data

 api:
   build: .
   container_name: fastapi_app
   ports:
     - "8000:8000"
   volumes:
     - ./app:/app/app
     - ./tests:/app/tests
   environment:
     - DATABASE_URL=postgresql://salva:salva@db:5432/productos
   # El comando para arrancar la API directamente
   command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
   depends_on:
     - db

volumes:
 postgres_data:
```

17 Django-marcador

Entender el archivo docker-compose.yml es entender cómo se comunican las piezas de tu infraestructura. Piensa en este archivo como el **director de orquesta**: él sabe qué músicos (servicios) deben tocar, en qué orden y cómo deben escucharse entre ellos.

Servicio 1: db (Base de Datos)

Este bloque levanta tu base de datos PostgreSQL.

- **image: postgres:15-alpine**: No descarga un sistema operativo pesado. Usa la versión **Alpine**, que es ultra ligera (pesa muy pocos MB), lo que hace que todo descargue y arranque más rápido.
- **container_name: postgres_db**: Es el nombre "real" del proceso en tu sistema. Si haces un docker ps, lo verás con este nombre.
- **restart: always**: Si la base de datos se cae o el ordenador se reinicia, Docker la levantará de nuevo automáticamente. Vital para la estabilidad.
- **environment**: Aquí defines las "llaves" de tu base de datos.
  - POSTGRES_USER y PASSWORD: Tu usuario y contraseña (salva/salva).
  - POSTGRES_DB: El nombre de la base de datos que se creará automáticamente al arrancar.
- **ports: "5432:5432"**: El primer número es el puerto de tu ordenador físico y el segundo es el del contenedor. Esto te permite conectar **PyCharm** o un gestor de bases de datos a localhost:5432.
- **volumes: postgres_data:/var/lib/postgresql/data**: **Esto es lo más importante.** Dice: "Todo lo que se guarde en la carpeta interna de Postgres, guárdalo también en un lugar seguro de mi disco duro llamado postgres_data". Si borras el contenedor, tus productos **no se borran**.

Servicio 2: api (Tu aplicación FastAPI)

Este bloque construye y ejecuta tu código Python.

- **build: .**: Le dice a Docker: "No busques una imagen en internet, constrúyela tú mismo usando el archivo Dockerfile que está en esta misma carpeta".
- **volumes**: Aquí hay un truco de desarrollo profesional:
  - ./app:/app/app: Mapea tu carpeta de código local con la del contenedor.
  - **¿Para qué sirve?** Para que cuando cambies una coma en PyCharm, el cambio se refleje **instantáneamente** dentro de Docker sin tener que reconstruir la imagen.

- **environment**:
  - DATABASE_URL: Fíjate que el host es @db:5432. Docker es listo: sabe que db se refiere al servicio de arriba. No necesitas usar IPs.
- **command**: Es el comando de arranque. El flag --reload es el que detecta los cambios que haces en el código y reinicia el servidor automáticamente.
- **depends_on: - db**: Le dice a la API: "No arranques hasta que el servicio db esté en marcha". Evita errores de conexión al iniciar.

El bloque final: volumes

YAML

```
volumes:
  postgres_data:
```

Aquí simplemente declaras formalmente que postgres_data es un volumen persistente gestionado por Docker. Sin esta línea al final, el archivo daría error.

Un detalle:

Fíjate que la API usa el puerto 8000:8000. Esto significa que:

1. **Desde fuera (tu navegador):** Accedes por localhost:8000.
2. **Desde dentro (los contenedores):** Se hablan entre ellos por sus nombres de servicio.

El archivo Dockerfile

```
# 1. Imagen base ligera y estable
FROM python:3.13-slim

# 2. Variables de entorno para optimizar Python en contenedores
# No escribe archivos .pyc y envía logs directamente a la consola
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# 3. Establecer el directorio de trabajo
WORKDIR /app
```

```
# 4. Instalar dependencias del sistema (si son necesarias)
# Limpiamos la caché de apt para mantener la imagen pequeña
RUN apt-get update && apt-get install -y --no-install-recommends \
   build-essential \
   && rm -rf /var/lib/apt/lists/*

# 5. Instalar dependencias de Python
# Copiamos solo el requirements primero para aprovechar la caché de capas
de Docker
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 6. Crear un usuario no-root por seguridad
# Por defecto, Docker corre como root, lo cual es un riesgo de seguridad
RUN useradd -m myuser
# Añade esto para que el usuario sea dueño de la carpeta de trabajo
RUN chown -R myuser:myuser /app
USER myuser
```

Si el archivo docker-compose.yml es el director de orquesta, el **Dockerfile** es el manual de entrenamiento de cada músico. Es el archivo que define exactamente qué hay dentro del "paquete" (la imagen) de tu aplicación para que, cuando se ejecute en cualquier parte del mundo, se comporte **exactamente igual** que en tu ordenador.

Vamos a desglosarlo punto por punto:

1. La base: FROM python:3.13-slim

Aquí eliges los cimientos. No usas un Python "pesado" (que incluiría herramientas que no necesitas), sino la versión **slim**. Es una imagen basada en Debian pero recortada al mínimo para que sea ligera y rápida de descargar, pero manteniendo la estabilidad de Python 3.11.

2. Optimización de Python: ENV

Estas dos variables son el "ajuste fino" para que Python se lleve bien con Docker:

- **PYTHONDONTWRITEBYTECODE=1**: Evita que Python genere archivos .pyc (archivos temporales). En un contenedor no los queremos porque ensucian la imagen y no aportan velocidad.
- **PYTHONUNBUFFERED=1**: Obliga a Python a enviar los mensajes (print, errores) directamente a la terminal sin guardarlos en un buffer.

¿Por qué es importante? Porque así, si la app falla, ves el error en los logs de Docker al instante.

3. El centro de operaciones: `WORKDIR /app`

Crea una carpeta llamada `/app` dentro del contenedor y se mete dentro. A partir de esta línea, todo lo que ocurra (copiar archivos, ejecutar comandos) sucederá dentro de esa carpeta. Es como hacer un `cd /app`.

4. Dependencias del sistema: `RUN apt-get`

A veces Python necesita herramientas del sistema operativo (como compiladores de C para instalar librerías complejas).

- **Truco DevOps:** El comando `rm -rf /var/lib/apt/lists/*` borra la lista de paquetes descargados. Como esa lista ya no se necesita después de instalar, borrarla hace que tu imagen final pese mucho menos.

5. La técnica del "Caché de Capas": `COPY requirements.txt`

Este es el paso más inteligente de tu archivo.

1. Copia **solo** el archivo de librerías.
2. Instala las librerías.
3. Más tarde (en el compose o en otro paso) se copia el resto del código.

¿Por qué se hace así? Docker guarda cada paso como una "capa" en memoria. Si mañana cambias una línea de tu código en `main.py` pero no añades nuevas librerías, Docker dirá: *"El requirements no ha cambiado, así que me salto el paso de instalar todo (que es lento) y uso lo que ya tenía guardado"*. Esto ahorra minutos de espera cada vez que programas.

6. Seguridad máxima: El usuario `myuser`

Por defecto, Docker ejecuta todo como "root" (el administrador total). Esto es peligroso: si alguien lograra hackear tu API, tendría el control total del contenedor.

- `useradd -m myuser`: Creas un usuario normal, con permisos limitados.
- `USER myuser`: Le dices a Docker: "A partir de ahora, ejecuta la aplicación con este guante de seda, no con el martillo de root". Es una práctica esencial para entornos de producción.

En resumen: ¿Qué hace este Dockerfile?

17 Django-marcador

Construye una **imagen blindada, ligera y rápida** que contiene solo lo necesario para que tu FastAPI brille, sin cargar con basura innecesaria y protegida contra intrusos.

El archivo conftest.py

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy.orm import sessionmaker
from app.main import app
from app.database import Base, engine, get_db

# Creamos una fábrica de sesiones para los tests
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)

@pytest.fixture(scope="session")
def setup_db():
    # Crea las tablas antes de los tests
    Base.metadata.create_all(bind=engine)
    yield
    # Las borra al terminar (opcional, si quieres ver los datos no lo
borres)
    # Base.metadata.drop_all(bind=engine)


@pytest.fixture
def db(setup_db):
    connection = engine.connect()
    transaction = connection.begin()
    session = TestingSessionLocal(bind=connection)

    yield session

    session.close()
    transaction.rollback()  # Esto hace que cada test empiece de cero
    connection.close()

@pytest.fixture
def client(db):
    # Inyectamos la sesión de prueba en la app
    def override_get_db():
        try:
```

```
            yield db
        finally:
            pass

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c
    # Limpiamos el override para no afectar a otros tests
    app.dependency_overrides.clear()
```

El archivo **conftest.py** es el **escenario de pruebas controlado**. Es el archivo que utiliza **Pytest** para configurar todo lo que tus tests necesitan antes de empezar a correr.

Su función principal es garantizar el **aislamiento**: que lo que pase en el Test A no afecte al Test B, y que tu base de datos no termine llena de basura tras las pruebas.

Una explicación detallada por bloques:

1. La Fábrica de Sesiones (TestingSessionLocal)

```
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
```

Aquí creas un "molde" para las sesiones de base de datos exclusivas para los tests. Aunque usa el mismo engine, nos permite generar conexiones independientes para que las pruebas no se choquen con la ejecución normal de la API.

2. Fixture setup_db: El Constructor

```
@pytest.fixture(scope="session")
def setup_db():
    Base.metadata.create_all(bind=engine)
    yield
    # Base.metadata.drop_all(bind=engine)
```

- `scope="session"`: Esto es vital. Significa que este bloque se ejecuta **solo una vez** cuando lanzas Pytest, no por cada test.
- `create_all`: Se asegura de que todas las tablas existan antes de empezar.
- `yield`: Divide la función en "antes del test" (crear tablas) y "después del test" (limpiar, si estuviera desmarcado el `drop_all`).

3. Fixture `db`: Aquí se hace el "Rollback"

Este es el corazón de la limpieza de datos:

```
@pytest.fixture
def db(setup_db):
    ...
    transaction = connection.begin()
    yield session
    transaction.rollback() # <--- LA CLAVE
```

Aquí ocurre la magia. Cada vez que un test pide la base de datos:

1. Se abre una **transacción** nueva.
2. El test hace lo que quiera (crear productos, borrarlos, etc.).
3. Al terminar el test, se ejecuta `transaction.rollback()`. **Resultado:** Los datos "desaparecen" de la base de datos real. Así, el siguiente test siempre encuentra la tabla vacía y lista.

4. Fixture `client`: El Infiltrado

```
@pytest.fixture
def client(db):
    def override_get_db():
        yield db
    app.dependency_overrides[get_db] = override_get_db
```

Esta es la parte más brillante de FastAPI. Tu aplicación real usa una función llamada `get_db` para conectarse a la base de datos. Con `dependency_overrides`, le dices a la aplicación:

> *"Oye, cuando alguien te pida la base de datos, no uses la de siempre; usa esta sesión de prueba (`db`) que acabo de crear".*

Esto permite probar la API **sin cambiar ni una sola línea de código** de tu archivo main.py.

Resumen: ¿Por qué es tan importante este archivo?

- **Automatización:** Crea las tablas por ti.
- **Limpieza:** Borra los cambios automáticamente tras cada test.
- **Flexibilidad:** Te permite usar un TestClient para simular peticiones HTTP (GET, POST) como si fueras un usuario real.

El archivo test_productos.py

test_main.py | test_productos.py

```python
from app.models import Producto

def test_verificar_persistencia_en_db(client, db):  Salvador Martos
    # 1. Definimos el producto
    payload = {
        "descripcion": "Monitor UltraWide 34",
        "precio": 350.00
    }

    # 2. Lo creamos a través de la API
    response = client.post("/productos/", json=payload)
    assert response.status_code in [200, 201]
    codigo_asignado = response.json()["codigo"]

    # 3. ¡LA PRUEBA REAL!: Consultamos la DB directamente sin usar la API
    # Buscamos en la tabla el producto con el código que nos dio la API
    producto_en_db = db.query(Producto).filter(Producto.codigo == codigo_asignado).first()

    # 4. Comprobamos que el objeto existe en la DB y los datos coinciden
    assert producto_en_db is not None
    assert producto_en_db.descripcion == "Monitor UltraWide 34"
    assert producto_en_db.precio == 350.00
    print(f"\n Confirmado: Producto {codigo_asignado} guardado en la base de datos.")

def test_crear_producto(client):...

def test_listar_productos(client):...

def test_crear_producto_sin_precio_falla(client):...

def test_crear_producto_precio_negativo_falla(client):...
```

Este test es potente porque usamos dos herramientas a la vez:

1. **client**: Actúa como un usuario externo que usa la web/app.
2. **db**: Actúa como un administrador que entra "por detrás" a la base de datos para verificar que el usuario no miente.

Cómo ejecutarlo y ver el mensaje: Para que veamos el `print` de confirmación en la consola, añade la bandera `-s` al comando:

```
docker-compose exec api pytest -s -p no:cacheprovider tests/
```

Un consejo para tu flujo de trabajo:

Ahora que tienes los tests funcionando, cada vez que hagas un cambio en el código (por ejemplo, si decides que el precio no puede ser negativo), solo tienes que darle a la flecha de arriba en tu terminal y ejecutar esto. Si sale **verde**, no has roto nada. **Esa es la tranquilidad que te da el DevOps.**

Explicamos alguna cosa:

Si observamos el contenido de estos archivos donde hay funciones como `test_verificar_persistencia_en_db` nos podríamos preguntar ¿desde dónde se llaman?, porque en Python "normal" las funciones no se ejecutan solas si no las llamas. Pero aquí entra en juego la magia de **Pytest**.

1. ¿Quién llama a la función?

   No la llamas tú, la llama **Pytest**.

   Cuando ejecutas `docker-compose exec api pytest`, el motor de Pytest escanea tu carpeta `tests/`, busca archivos que empiecen por `test_*.py` y, dentro de ellos, busca funciones que empiecen por `test_*`. Al encontrarlas, las mete en una "cola" y las ejecuta una a una.

2. ¿Cómo funciona el módulo de test? (El truco de la inyección)

   Aquí es donde ocurre lo que llamamos **Inyección de Dependencias**.

   Fíjate que la función dice `def test_verificar_persistencia_en_db(client, db):`.

   1. Pytest ve que necesitas `client` y `db`.
   2. Va al archivo `conftest.py` y busca las funciones que tengan el decorador `@pytest.fixture` con esos mismos nombres.
   3. Ejecuta primero la lógica de la fixture (preparar la base de datos).
   4. Te "pasa" esos objetos ya listos a tu función de test.

En resumen, el flujo es:

1. **Pytest** detecta el test.

2. **Fixture** abre una transacción en la DB.
3. **Tu Test** le pide a la API que cree un monitor.
4. **Tu Test** mira en la DB si el monitor está ahí (y lo ve, porque la transacción sigue abierta).
5. **Fixture** hace el rollback (el monitor desaparece de la DB real).
6. **Pytest** te da el verde.

---

17 Django-marcador

El archivo \.github\workflows\ci.yml

17 Django-marcador

```yaml
name: API CI/CD Pipeline

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      # Levantamos la DB en GitHub para los tests
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_USER: salva
          POSTGRES_PASSWORD: salva
          POSTGRES_DB: productos
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Descargar código
        uses: actions/checkout@v4

      - name: Configurar Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Instalar dependencias
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Ejecutar Tests
        env:
          # Le decimos a los tests dónde está la DB de GitHub
          DATABASE_URL: postgresql://salva:salva@localhost:5432/productos
        run: |
          # Usamos PYTHONPATH=. para que encuentre la carpeta 'app'
          export PYTHONPATH=$PYTHONPATH:.
          pytest -p no:cacheprovider tests/
```

El archivo ci.yml es el **corazón de tu automatización.** Es el guión que sigue **GitHub Actions** para montar un servidor desde cero cada vez que subes código, probarlo y decirte si es seguro o si vas a romper algo en producción.

El desglose paso a paso de lo que está ocurriendo "en la nube":

1. El Disparador (on)

YAML

```yaml
on:
  push:
    branches: [ main, master ]
```

Esto le dice a GitHub: "No trabajes por gusto". El robot sólo se despierta cuando haces un git push o abres un pull request hacia las ramas principales. Si trabajas en una rama secundaria, el robot descansa.

2. El Escenario (runs-on: ubuntu-latest)

GitHub nos presta una máquina virtual limpia con **Ubuntu Linux,** asegurando que no haya "basura" de ejecuciones anteriores que falsee los tests.

3. El "Sidecar" de Base de Datos (services)

Esta es una de las partes más potentes:

YAML

```yaml
services:
  postgres:
    image: postgres:15-alpine
    ...
    options: >-
      --health-cmd pg_isready ...
```

Como tu API necesita una base de datos para los tests, GitHub levanta un contenedor de **Postgres** en paralelo.

- **Health-check:** El comando pg_isready es vital. Asegura que los tests no empiezan hasta que la base de datos esté "despierta" y lista para recibir conexiones. Sin esto, los tests fallarían por intentar conectar demasiado pronto.

---

4. Los Pasos (steps)

17 Django-marcador

Aquí es donde ocurre la acción secuencial:

1. **actions/checkout@v4**: El robot "clona" tu código de GitHub dentro de la máquina virtual. Sin esto, la máquina estaría vacía.
2. **setup-python**: Instala la versión exacta de Python (3.13) que usas en el proyecto.
3. **pip install**: Instala todas las librerías de tu requirements.txt.
4. **Ejecutar Tests**:
    - **DATABASE_URL**: Le pasamos la dirección de la base de datos que GitHub levantó en el paso de services.
    - **export PYTHONPATH=$PYTHONPATH:.**: Un truco técnico necesario para que Python sepa dónde está la carpeta app/ y no se pierda al importar los módulos.
    - **pytest**: El momento de la verdad. Si todos los tests dan verde, GitHub te pone la validación en el commit.

¿Por qué este archivo te hace mejor desarrollador?

- **Dormir tranquilo:** Sabes que si te olvidas una coma o rompes una lógica, el ci.yml te avisará antes de que el error llegue al cliente.
- **Documentación viva:** El archivo explica exactamente qué necesita el proyecto para funcionar (Python 3.13, Postgres 15, etc.).
- **Ahorro de tiempo:** No tienes que pasar los tests en local manualmente cada vez; el robot lo hace por ti mientras tú sigues programando.

Un detalle:

Fíjate que en DATABASE_URL usamos localhost:5432. En el docker-compose.yml local usábamos db:5432. Esto es porque en GitHub Actions, los servicios corren en la "red del host" de la máquina virtual, por lo que se ven a través de localhost.